

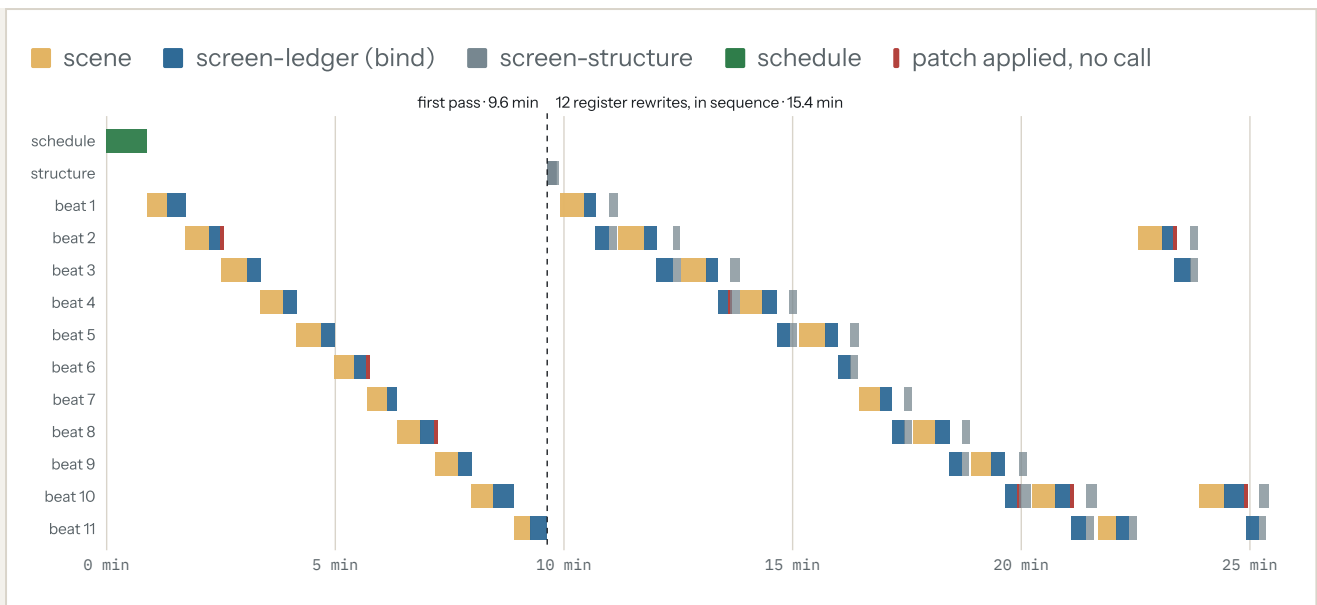
The Ten-Minute Loop

A change to a drafting prompt takes 35 to 45 minutes and about \$26 to judge today, and most of the steps are by hand. Three moves bring a scene-level change to a judged result in about five minutes, and a schedule-level change in about twelve: branch a draft at a fixed point, run rewrites and judges in parallel, and make the protocol a command. The prompts themselves stay as they are until that loop says otherwise.

TODAY · ONE EXPERIMENT 35–45 min three drafts a side, 36 judge passes, by hand	TODAY · COST ≈ \$26 3 × \$7.67 drafts + ≈ \$3 of judging	PROPOSED · SCENE CHANGE ≈ 5 min beat-level A/B, ≈ \$5, schedule pinned
PROPOSED · SCHEDULE CHANGE ≈ 12 min three parallel drafts a side, ≈ \$25		

1 · WHERE THE MINUTES GO

The store holds one twelve-call-per-beat draft with full timing: 20260921145735-c933 , eleven beats under the signal profile, 25.4 minutes from the schedule call to the last screen, \$12.40. The chart below is drawn from its 100 step rows to scale. The first pass, one scene then one ledger bind per beat in sequence, takes 9.6 minutes. The register rewrites that follow take 15.4 minutes, because registerRewrites runs them one at a time in beat order, and each one is a scene call, two binds and two structure screens.



Rows are beats; the schedule and the parallel structure pass sit on their own rows. The dashed line marks the end of the first pass at 9.6 minutes. Everything to its right is twelve register rewrites, run serially.

STAGE	CALLS WITH USAGE	MEAN S	MEAN \$	THINKING TOK	CACHE HIT, SINCE 21 SEP	NOTE
scene	102	28	0.26	271	100%	on the critical path once per beat
screen-ledger	142	39 (18 recent)	0.12	2,551 (1,116 recent)	100%	on the critical path once per beat; 83% of its output was thinking on run 10
screen-structure	129	13	0.05	410	97%	parallel across beats
schedule	7	63	0.21	1,693	100%	one call; its variance was the only real difference in the 22 Sep head-to-head
check-ledger / check-derivation	46 / 46	100 / 97	0.27 / 0.25	7,906 / 6,894	100%	a check pass is 1.1 to 2.8 min and \$1.2 to \$2.5
outline	6	69	0.18	2,155	100%	ideation is 3 to 13 min a brief
report.pdf	—	≤ 60	0	—	—	a headless Chrome print, awaited by <code>draft</code> and every <code>rewrite</code>

Means over every step row with a `usage` column (684 of 6,918; the column arrived 20 Sep). The three listen-profile arms judged on 22 and 23 Sep ran in scratch stores and are not in the live store; their write-ups give 27 to 36 minutes and \$6.84 to \$9.13 a draft, 82 to 103 calls, 2 to 11 register rewrites.

2 · THE LOOP TODAY, STEP BY STEP

STEP	HOW IT RUNS NOW	WALL	HAND WORK
land the change	worktree, squash to main; the unit rebuilds	1–2 min	none
a baseline to compare against	three drafts of one brief by the old code, in a copied store, served on port 80 for <code>judge.py</code> to fetch	≈ 31 min, once	copy the store, serve it, remember the ids

three drafts a side	<code>cloudchamber draft --profile listen</code> three times; each draft re-derives its schedule	27–36 min, parallel if stores differ	copy stores; two CLI draws on one store lock it
judge	<code>judge.py</code> once per pair per judge, 4 passes each, passes in sequence inside a run; 9 runs plus a floor run	unmeasured, $\approx$ 5–10 min	nine invocations, output files named by hand
pool	<code>tally.py</code> over the json files	seconds	none
write up	one <code>evals/<run>.md</code> , 25 so far since 15 Sep	10–20 min	all of it; the tables are typed

Two things in that table cost more than the minutes. The schedule is re-derived per draft, so a scene-prompt change is judged through the schedule's own variance; the 22 Sep head-to-head found the one real difference between arms was a schedule sample, not the change. And nothing stores what a judge said: the verdicts live in json files in a scratch directory and in prose in `evals/`. Judgement is the one thing the store cannot regenerate, which is the argument ADR-0002 already makes for `bank/verdicts.jsonl`.

3 · THE EVALUATION LADDER

Not every change needs a full draft. The pipeline already has natural units smaller than a story, each one call: a beat, a schedule, a check pass. The ladder names what each level can and cannot detect, so a change is judged at the cheapest level that can see it.

L0

Deterministic, on stored text. The listen profile, the slop screen, the restated screen, the rewrite count, the cost and the cache-hit rate. Seconds, \$0. Sees: a register rewrite blow-up like the presence run's 10 → 25, a long-share or numeral drift, a cost regression. Cannot see quality. Exists today; needs to be reported as one row per draft.

L1

Beat-level A/B on a pinned draft. Take a stored draft. Fix its brief, ledger, schedule and story-so-far. Redraft K beats per arm (the hook beat, the paying beat, the last beat, two more), N samples each; every call is independent, so all run at once. Screen them. Judge beat pairs blind, both orders, three outside judges, on a beat rubric. $\approx$ 5 min, $\approx$ \$5 at K=5, N=3. Sees: register and scene-ask changes, the screens' lines, effort or model changes on scene and bind. Cannot see: anything the schedule decides, drift across a whole story.

L2

Full draft A/B on a pinned brief. Three drafts a side from one brief, matched pairs, both orders, the within-arm floor: the protocol the 22 and 23 Sep runs converged on, run in parallel by a command. $\approx$ 12 min after candidates 4 and 5, $\approx$ \$25. Sees: schedule-ask changes, template and profile changes, chronology, cast. The floor run costs as much again and is owed once per baseline, not per experiment.

L3

Chain-level. Ideate, check, repair and draft from a seed. Hours. Sees: premise, outline and checker changes. Run rarely, on a fixed seed set. Not in scope for the ten-minute target.

The beat rubric is the one open design question. The eight axes are story axes; hook, ending and momentum belong to particular beats or to the whole. A beat rubric would keep presence, people, feeling, cost and clarity as they are, ask hook only of beat 1 and ending only of the last beat, and drop momentum. Whether beat-level wins predict story-level wins is itself the first L2 run to make with the new loop: judge the same arms both ways once.

4 · CANDIDATES

In the order to build them. Effort is my time, not a person's. `judged run owed` marks a change that alters what a model reads, and so lands only through the protocol; `schema` marks a store change.

01 Branch a draft at a point

`strong` `schema 13` ≈ 30 min the substrate for 02 and 03

A new draw row that develops an existing draft from a chosen step: `cloudchamber branch <draw> [--at-beat K] [--models ...] [--profile ...]`. It copies the brief parts as a fork does today, copies the schedule step and the bound scenes 1..K-1 as `copied` steps, then runs `draft` from beat K. With no `--at-beat` it copies the schedule alone and writes every scene again: two arms drafted from one schedule.

- `draws.branched_from` and `draws.branch_at` (a step id); `Lineage.path` follows the new link as it follows `forked_from`; `chain.scenes()` already takes the latest artifact per beat, so copied scenes read as written.
- The same primitive answers Chris's ask directly: redraft from beat 7 under a different model, with everything before it held still.
- What it treats as fixed: one schedule per draw. A branch is a new draw, so the rule holds.

Pinning the schedule is the largest variance cut available. The head-to-head run found the only real difference between two arms was the schedule's chronology choice, and the replication run needed three drafts a side to see past it.

02

The experiment runner

strong

bank/judgements.jsonl ≈ 90 min

One command runs the protocol: `cloudchamber lab <brief> --level L1|L2 --arm main --arm <ref-or-worktree> [--beats 1,6,9,12] [--n 3]`. It copies the store once per arm into a scratch directory, runs each arm's drafts or beats in parallel, runs every judge pass at once across the panel, pools with the floor, writes the tables and the JSON, and appends every verdict to `bank/judgements.jsonl`. The write-up in `evals/` becomes a rendered file with a three-sentence reading typed at the end, not a document typed from scratch.

- `judge.py` runs its passes in sequence and one pair per process. The runner spawns the passes concurrently and takes texts by draw id from the scratch store, not from port 80.
- A `judgements` table replayed from the log, one row per pass: experiment, arm, pair, judge, order, the eight axes, overall, cost. `tally.py`'s weight and floor rules move into the runner; the script stays as the reference implementation until the numbers match.
- The floor is cached per baseline: a within-arm pair judged once serves every experiment on that brief until the baseline moves.

Fixed: the panel, the both-orders rule, the weight by order-following, the floor. Nothing in the protocol changes; it stops being typed.

03

Beat-level judging

strong

≈ 45 min needs O1

L1 in the runner: branch the baseline draft at each chosen beat, N times per arm, judge each arm-pair of beats blind with the story-so-far shown once as context above both. The beats a change aims at are chosen by hand (`--beats`); the default is beat 1, the paying beat, the last beat and two picked at random.

- The judge prompt for a beat shows the schedule entry for that beat and the previous scene, then the two candidate scenes. The judge answers the beat axes and an overall.
- The first run with it is a calibration: the three presence-run drafts against the three fix drafts, judged at L1 on their paying beats. If L1 does not reproduce the L2 loss, L1 is for register changes only and the page says so.

04 Take the rewrites and the report off the critical path

strong

judged run owed for the rewrite order ≈ 20 min

Two changes to `Drafting.draft`. The PDF print stops being awaited: the report is written when the draft reaches gate 2, and the PDF is printed in the background or on `keep`. Up to sixty seconds a draft and a rewrite, no model reads it, no run owed.

The register rewrites run in two waves, odd beats then even, instead of one at a time in beat order. On c933 this is 15.4 minutes to about 3. It changes one input: a rewritten odd beat's story-so-far holds the original even beat before it where the serial order held the rewritten one. That is a prompt-input change, so it lands through L2 once the runner exists, and the runner is what it makes affordable.

05 Write the next beat while the last one binds

worth exploring

judged run owed ≈ 30 min

The first pass is a chain of scene, bind, scene, bind: about 55 seconds a beat, 9.6 minutes for eleven. Scene K+1 could start on the raw text of scene K while the bind of K runs, and the bind of K+1 then holds it to the bound K, which is exactly what `rewrite k` does today for the beat after it. The pass would take about 30 seconds a beat.

The cost is that K+1 may inherit a contradiction the bind of K would have patched. The store says how often that matters: 258 of 558 scene steps are patched after their bind, 46%, and the patches are span substitutions. Whether the inherited half changes what the panel scores is an L2 question, and a cheap one once 02 exists.

Alternative with no judged run: `effort = "low"` on `screen-ledger` in `stages.toml`. The knob exists and nothing sets it. The 22 Sep probe found medium on the checks changed nothing; low on the bind is unmeasured. Recent binds already run 18 s, so the ceiling on this is small.

06 Provenance and a timeline for every draw

strong

schema 13

≈ 40 min

Three things the store does not record and every eval has had to reconstruct.

`steps.version` : the git sha and a dirty flag, as `pipelineVersion()` already computes for verdicts. `steps.templates` : a hash of each prompt template the step filled, so "which prompt wrote this scene" is a query. And `cloudchamber timeline <draw>` : the waterfall above as text, from the columns already stored, with the critical path marked.

- One metrics row per draft, on the draw JSON and the report: wall by phase, cost by stage, calls, register rewrites, patches applied, cache-hit rate, flags by screen, the listen profile. The lab reads the same row for its L0 column.
- Optional: emit each draw as a trace to the Langfuse at 127.0.0.1:3010, one span per step. The OTLP endpoint takes it with no other dependency, and the waterfall is then a page rather than a command.

07 One table of constraints, byte for byte the same prompts

worth exploring

≈ 60 min

no judged run: a snapshot test proves the prompts unchanged

The vault says not to build a constraint registry, because merging a register sentence with its rewrite line changes what the model reads. That warning is right and this candidate does not merge anything. It co-locates: one row per constraint with its four moments, the schedule clause, the register sentence, the screen question and the rewrite line, each kept verbatim, and the templates assembled from the rows. A snapshot test asserts every filled template is byte-identical to today's.

What that buys is not shorter prompts. It is an ablation switch per row, `--without one-voice`, that the lab can drive, so the question CLAUDE.md asks of every clause is answered by a five-minute run instead of a branch. The five sentences of `sceneSignal` that restate a screen become five rows with a flag each, and the caps ablation becomes one afternoon of L1 runs. Deduplication, if any, comes after those runs, one row at a time.

worth exploring, later

every step a judged run

`scheduleTold`, `scheduleSignal` and `scheduleListen` say the same things in different words: a cost paid by a named person before the last beat, a set piece per beat, the last beat never the shortest, a cast. Signal and listen share about two thirds of their text. In code the three carry special cases: the told template's beat-1 replay is exempt from the restated screen; the paying beat defaults to $M-1$ under a shaped template and M otherwise; the register is inferred from container and template when set to auto; the four paying-beat questions map onto two rewrite lines.

The generic shape is the one the listen template already states: requirements, with the form left to the brief, plus an optional channel shape. Whether told and signal can be that plus one line each is a question for the loop, not for a refactor. Recommendation: do nothing to the prompt text now; list each special case as a lab run; land what the panel does not miss.

quality ≈ 60 min in total

- `Drafting` (471 lines) holds gate 1, the auto-repair loop, gate 2 and the findings view. The auto loop and reconcile are their own module; `rewritePlan` and the register lines belong beside `STRUCTURE_RULES` in `write.ts`.
- Tunables outside any toml, so `help --md` does not list them: `CLEAN_PASSES`, `VERIFY_READINGS`, `AUTO_CHECKERS`, `RUN.cacheLeadMs`, `INVALIDATES_WEIGHT`, `HEDGED`. Move the numeric ones under `[repair]` and `[checks]` in `draft.toml`.
- The 20 Sep review's item 06 is still open: `structure.template = "auto"` by default makes the `[screens.listen]` ceilings dead config on a default draft, and the ceilings are untyped. Type them and say so in the toml.
- 32 exports in `write.ts`, `drafting.ts` and `check.ts` with no consumer in another module, 17 with none at all (the 23 Sep note).
- The restated screen has raised 616 flags and carries no patch; nothing acts on one except a hand `rewrite k`. Either it gets a line in the rewrite plan under a judged run or it moves to the L0 report as a count.
- The test suite is 81 s wall for 14 s of CPU; `drafting.test.ts` is 57 s for 75 tests, about 0.75 s of fixture each. A shared fixture per describe, or files run in parallel, puts the suite under 20 s, which the loop wants.
- 93 MB of prompt text in `steps`: the same brief block or scene context is stored on every step of a pass. A content-addressed `blobs` table cuts the store by about half and makes "which steps shared a system prompt" a join. Optional; the store is on ext4 now and reads fast enough.

5 · SPECIAL CASES, AND WHERE EACH WOULD BE SYSTEMATIZED

WHERE	SPECIAL CASE	GENERIC FORM	HOW IT LANDS
<code>scenesession.ts:187</code>	told template: a repeat of beat 1 is not a restated flag	a schedule beat can declare <code>replays: 1</code> ; the screen reads the schedule, not the template name	code change, no prompt change; snapshot
<code>scenesession.ts:169</code>	paying beat defaults to M-1 under a shaped template, M under auto	the schedule always marks <code><pays></code> ; a schedule that does not fails shape	<code>judged</code> the ask changes for auto
<code>write.ts:149</code>	register inferred from container and template when auto	the profile names its register; <code>auto</code> goes	config change; the three profiles already name one or imply one
<code>write.ts:62-65</code>	four paying-beat questions, two rewrite lines	one question per line, or one line per question	<code>judged</code> L1 on the paying beat
<code>write.ts:216</code>	theme-stated is allowed on the last beat	a rule row carries <code>except: last</code>	code change; the table already has <code>asked</code>

<code>prompts.ts</code> <code>sceneSignal</code>	five of eight sentences restate a screen (length, numerals, voices, time moved, body)	each is one constraint row with a switch	07, then one L1 per row
<code>drafting.ts:60-62</code>	<code>AUTO_CHECKERS</code> , <code>CLEAN_PASSES</code> as constants	keys under <code>[repair]</code>	config change
<code>recur.ts:181</code>	the score: six terms, 35% of findings at exactly 6 or 7	leave it; the 20 Sep measurement says every term decides a third of the findings	not a cleanup; an L3 question
<code>model.ts:91</code>	<code>tag()</code> reads <code><parameter name=...></code> as the tag (Sonnet 5)	keep; the parser is the right layer for output variance	none

6 · WHAT STAYS FIXED

Every candidate above keeps the decisions in `docs/adr/` : model calls are sealed `claude - p` subprocesses (0001); the log in `bank/` is the source of truth and the store is a replay (0002, which the judgements log follows); no model decides at a gate (0006); findings rank by recurrence (0008); a repair is a new draw held to a pinned ledger (0009); fixed context rides in the system prompt word for word (0010, which is why branching copies the schedule step rather than re-filling it); drafts are judged by other families, both orders (0011).

The one probe that would move an ADR is the session-carry idea from 22 Sep, which needs session persistence and is not proposed here.

7 · ORDER OF WORK, AND THE LOOP TIME AFTER EACH STEP

1. **06 provenance and timeline** first, because every later step is measured with it. Loop unchanged.
2. **01 branch**, then **04** report off the path. A schedule-pinned L2 is possible by hand: drafts fall to about 20 min on a twelve-beat listen brief.
3. **02 runner** at L2 only, with the judgements log. Land-to-tables $\approx$ 25 min, no hand steps.
4. **03 beat-level**, and the calibration run. Scene-level changes $\approx$ 5 min.
5. **04** rewrite waves and **05** pipelined binds, each landed through the runner. Schedule-level changes $\approx$ 12 min.
6. **07 constraint table**, then the first ablation series: the five restating sentences of `sceneSignal` , one L1 each.
7. **09** as the runs go, and **08** only with results in hand.

Estimated total for 01 to 07: about five hours of my time across sessions, plus the judged runs, which are about \$5 for each L1 and \$25 for each L2 at list price.

8 · TO SETTLE BEFORE STARTING

- **The beat rubric.** Keep the eight story axes and ask each only where it applies, or write beat axes? I recommend the former, with hook on beat 1 and ending on the last beat, and let the calibration run decide.
- **The floor's home.** One within-arm pair per baseline brief, cached and reused, or one per experiment as now? I recommend per baseline; the 23 Sep note says the floor must share the comparison's judges, not its date.
- **Rewrite waves and pipelined binds** are wall-clock wins that change what a model reads. Land through L2 (my recommendation) or accept on the cost figures alone?
- **Langfuse or a command** for the timeline. The command costs nothing to keep; the traces give a page. Both is fine.

Sources: the live store at `~/ .cloudchamber/cloudchamber.db` read-only (6,918 steps, 684 with usage; c933 timeline of 100 steps); `evals/20260922-*.md` and `20260923-*.md` ; the vault notes on the 17, 19 and 20 Sep reviews and the 23 Sep runs; the code at b7830bf. Nothing was changed, and no model call was made.